Algorithmique I

Chapitre 5 · Itérations

Corrigé des exercices (2026-2027)

[1, pp. 49-56]. Dans les parcours de chiffres, on travaille sur la valeur absolue avant d'appliquer `DIV` et `MOD`. Les boucles `for` ont des bornes inclusives.

Exercice 1 : Les quatre traces

Syllabus, p. 49–50. Un tiret désigne une valeur encore indéterminée. Aucun des quatre algorithmes n'affiche de résultat.

a) Ligne 5, valeur de n après chaque affectation. Après la ligne 3, $n = 1$. À chaque entrée dans le corps, i prend la valeur suivante :

i	1	2	3	4	5
n après la ligne 5	3	8	19	42	89

Après la boucle, $n = 89$; on ne réutilise pas la valeur finale de i.

b) Lignes 6 et 7, dans cet ordre. Après les lignes 3 et 4, $p = 1$ et $s = 0$.

i	5	4	3	2	1	0
p utilisé à la ligne 6	1	2	4	8	16	32
s après la ligne 6	1	3	7	15	31	63
p après la ligne 7	2	4	8	16	32	64

c) Lignes 6 à 8. État initial après la ligne 4 : $(n, c, s) = (20, –, 0)$.

Tour	1	2	3
n avant le tour	20	6	2
c après la ligne 6	2	0	2
s après la ligne 7	2	2	4
n après la ligne 8	6	2	0

Le test ligne 5 échoue ensuite : le dernier n vaut zéro.

d) Lignes 8 à 10. État initial après la ligne 6 : `ok` $=$ `true`, $n = 100$, `somme` $= 0$.

Tour	1	2	3	4	5
n avant le tour	100	50	25	12	6
`somme` après la ligne 8	100	150	175	187	193
n après la ligne 9	50	25	12	6	3
`ok` après la ligne 10	vrai	vrai	vrai	vrai	faux

Le nombre 3 sert au dernier test, mais n'est pas ajouté.

Exercice 2 : Affichage

Syllabus, p. 50. Les cinq variantes sont indépendantes. En (e), 1 est 2^0.

```
void afficherA()
    int i
    for i from 1 to 10
        print i

void afficherB()
    int i
    for i from 10 to 1 by -1
        print i

void afficherC()
    int i
    for i from 2 to 100 by 2
        print i

void afficherD()
    int i
    for i from 1 to 91 by 10
        print i

void afficherE()
    int exposant
    for exposant from 0 to 10
        print 2**exposant
```

Exercice 3 : Affichage avec un paramètre

Syllabus, p. 50. En (d), on parcourt les multiples naturels, donc 0 est compris si $n > 0$. Si seuls les multiples strictement positifs sont demandés, démarrer à 5. En (f), on prend la suite usuelle $1, 4, 9, \ldots$; si la convention de départ inclut 0^2, remplacer `i**2` par `(i-1)**2`. Un n non positif ne produit aucun terme.

```
void afficherA(int n)
    int i
    for i from 1 to n
        print i

void afficherB(int n)
    int i
    for i from n to 1 by -1
        print i

void afficherC(int n)
    int i
    for i from 1 to n
        print 2*i - 1

void afficherD(int n)
    int i
    for i from 0 to n - 1 by 5
        print i

void afficherE(int n)
    int i
```

```
    for i from 1 to n
        print 5*i

void afficherF(int n)
    int i
    for i from 1 to n
        print i**2
```

Exercice 4 : Quatre suites

Syllabus, p. 50. En (a), on ajoute le rang au total précédent. En (c), les écarts alternent entre 1 et 2. En (d), une variable auxiliaire conserve le prochain terme avant de décaler le couple.

```
void afficherA(int n)
    int i, terme
    terme = 0
    for i from 1 to n
        terme = terme + i
        print terme

void afficherB(int n)
    int i, terme
    terme = 1
    for i from 1 to n
        print terme
        terme = -terme

void afficherC(int n)
    int i, terme
    terme = 1
    for i from 1 to n
        print terme
        terme = terme + 2 - (i MOD 2)

void afficherD(int n)
    int i, a, b, suivant
    a = 0
    b = 1
    for i from 1 to n
        print a
        suivant = a + b
        a = b
        b = suivant
```

Exercice 5 : Statistiques sur les interros

Syllabus, p. 51. Un seul parcours répond à (a)–(e). Quand un nouveau maximum apparaît, sa fréquence repart à 1. La sentinelle n'est pas une cote. Si aucune cote n'est lue, les deux comptes valent zéro et la moyenne et le maximum ne sont pas définis.

```
void statistiques()
    int cote, nb, nbNulles, somme, meilleure, nbMeilleures
    nb = 0
    nbNulles = 0
    somme = 0
    meilleure = 0
```

```
    nbMeilleures = 0
    read cote
    while cote ≠ -1
        nb = nb + 1
        somme = somme + cote
        if cote == 0
            nbNulles = nbNulles + 1
        if nb == 1 OR cote > meilleure
            meilleure = cote
            nbMeilleures = 1
        else if cote == meilleure
            nbMeilleures = nbMeilleures + 1
        read cote
    print "Nombre de cotes :", nb
    print "Cotes nulles :", nbNulles
    if nb > 0
        print "Moyenne :", somme / nb
        print "Meilleure cote :", meilleure
        print "Nombre d'apparitions :", nbMeilleures
    else
        print "Aucune cote : ni moyenne ni maximum."
```

Contrôle : $0, 16, 16, 0, -1$ donne 4 cotes, dont 2 nulles, une moyenne de 8 et un maximum de 16 atteint deux fois.

Exercice 6 : Évolution d'un capital

Syllabus, p. 51. L'année 0 affiche le montant de départ. On applique le taux au capital courant, intérêts précédents compris. Arrondir au centime à l'affichage seulement.

```
void capitalise()
    float capital, taux
    int n, annee
    read capital, taux, n
    print 0, capital
    for annee from 1 to n
        capital = capital * (1 + taux / 100)
        print annee, capital
```

À 2%, 1000 donne 1020, 1040,40, puis 1061,21 après arrondi d'affichage.

Exercice 7 : Le prix d'entrée

Syllabus, p. 51. Toutes les personnes saisies comptent pour la remise, même les entrées gratuites. L'âge zéro est réservé à la fin de la saisie et ne peut pas représenter un membre du groupe.

```
void prixGroupe()
    int age, nb
    float total
    nb = 0
    total = 0
    read age
    while age ≠ 0
        nb = nb + 1
        if age < 6
            total = total + 0
        else if age ≤ 12
```

```
            total = total + 8
        else if age ≤ 18
            total = total + 15
        else if age ≤ 60
            total = total + 25
        else
            total = total + 15
        read age
    if nb ≥ 10
        total = total * 0.75
    print total
```

Contrôle : neuf adultes à 25 euros paient 225 euros ; avec une dixième personne de cinq ans, le groupe paie 168,75 euros.

Exercice 8 : Y a-t-il un zéro ?

Syllabus, p. 52. `MOD 10` isole le dernier chiffre et `DIV 10` le retire. En (a), le nombre 0 lui-même contient le chiffre zéro s'il est admis parmi les entrées. En (b), un entier qui contient plusieurs zéros ne compte qu'une fois.

```
Boolean contientZero(int n)
    Boolean trouve
    trouve = (n == 0)
    while n > 0 AND NOT trouve
        trouve = (n MOD 10 == 0)
        n = n DIV 10
    return trouve

void compterAvecZero()
    int n, nb
    nb = 0
    read n
    while n ≠ -1
        if contientZero(n)
            nb = nb + 1
        read n
    print nb
```

Exercice 9 : Le prof clément

Syllabus, p. 52. La première cote existe et initialise le minimum. Avec plusieurs cotes, on soustrait le minimum *une fois*, même s'il apparaît plusieurs fois.

```
void afficherCoteFinale()
    int cote, nb, somme, minimum
    float resultat
    read cote
    minimum = cote
    somme = cote
    nb = 1
    read cote
    while cote ≥ 0
        nb = nb + 1
        somme = somme + cote
        if cote < minimum
            minimum = cote
```

```
        read cote
    if nb == 1
        resultat = somme
    else
        resultat = (somme - minimum) / (nb - 1)
    print resultat
```

Contrôles : $15, 8, 12, 6, 17, 13$ donne 13 ; 6 seul donne 6 ; $5, 5, 14$ donne 9,5.

Exercice 10 : Le tyrannosaure

Syllabus, p. 52–53. L'état mémorise le suffixe déjà lu qui peut encore devenir `AGCC` : 0 pour aucun, 1 pour `A`, 2 pour `AG`, 3 pour `AGC`. Un nouveau `A` lance un motif même en cas d'échec du précédent. Après un motif complet, aucun de ses suffixes propres n'est un préfixe du motif.

```
void identifierADN()
    String base
    int etat, nb
    etat = 0
    nb = 0
    read base
    while base ≠ "X"
        if base == "A"
            etat = 1
        else if etat == 1 AND base == "G"
            etat = 2
        else if etat == 2 AND base == "C"
            etat = 3
        else if etat == 3 AND base == "C"
            nb = nb + 1
            etat = 0
        else
            etat = 0
        read base
    if nb == 177
        print "ADN du tyrannosaure"
    else
        print "Pas l'ADN du tyrannosaure"
```

L'exemple du syllabus contient trois occurrences. Deux motifs contigus comptent deux fois ; 176 et 178 occurrences sont toutes deux refusées.

Exercice 11 : Factorielle

Syllabus, p. 53. Domaine : $n \geq 0$. Le produit vide vaut 1, ce qui couvre $0!$.

```
int factorielle(int n)
    int i, produit
    produit = 1
    for i from 2 to n
        produit = produit * i
    return produit
```

Exercice 12 : Le premier chiffre

Syllabus, p. 53. Retirer les derniers chiffres jusqu'à n'en garder qu'un. Le signe ne compte pas comme chiffre.

```
int premierChiffre(int n)
    if n < 0
        n = -n
    while n ≥ 10
        n = n DIV 10
    return n
```

Ainsi 23540 donne 2 et 0 donne 0.

Exercice 13 : Le nombre miroir

Syllabus, p. 53. Chaque chiffre extrait s'ajoute à droite du miroir en construction. Le résultat entier ne conserve pas les zéros de tête : 1200 devient 21. Pour un entier négatif, on conserve le signe.

```
int miroir(int n)
    int signe, resultat
    signe = 1
    if n < 0
        signe = -1
        n = -n
    resultat = 0
    while n > 0
        resultat = 10*resultat + (n MOD 10)
        n = n DIV 10
    return signe * resultat
```

Exercice 14 : La somme des chiffres

Syllabus, p. 53. On additionne les restes de divisions par 10. Pour un entier négatif, on additionne les chiffres de sa valeur absolue.

```
int sommeChiffres(int n)
    int somme
    if n < 0
        n = -n
    somme = 0
    while n > 0
        somme = somme + (n MOD 10)
        n = n DIV 10
    return somme
```

Exercice 15 : Nombre réduit

Syllabus, p. 53. La réduction s'applique à la valeur absolue et réutilise `sommeChiffres` (exercice 14). Pour $n \geq 10$, la somme de ses chiffres est strictement inférieure à n, ce qui garantit l'arrêt.

```
void afficherReduction()
    int n
    read n
    if n < 0
        n = -n
    while n ≥ 10
        n = sommeChiffres(n)
    print n
```

Contrôle : $468605 \to 29 \to 11 \to 2$.

Exercice 16 : Le PGCD

Syllabus, p. 54. Euclide conserve $\mathrm{PGCD}(a, b) = \mathrm{PGCD}(b, a \bmod b)$. Mémoriser le reste avant de remplacer a. Si $0 \leq a < b$, le premier reste vaut a : le premier tour échange alors les deux valeurs. Il n'y a pas de traitement spécial.

```
int pgcd(int a, b)
    int reste
    if a < 0
        a = -a
    if b < 0
        b = -b
    while b ≠ 0
        reste = a MOD b
        a = b
        b = reste
    return a
```

$\mathrm{PGCD}(112, 63) = \mathrm{PGCD}(63, 112) = 7$ et $\mathrm{PGCD}(0, 9) = 9$. Le couple $(0, 0)$ donne 0 par convention informatique.

Exercice 17 : Le PPCM

Syllabus, p. 54. On utilise les valeurs absolues et on adopte $\mathrm{PPCM}(a, 0) = 0$.

a) Recherche de multiples. On avance par pas égal au *plus grand* des deux nombres pour énumérer moins de candidats avant le PPCM. Avec $a = 6$ et $b = 15$, on teste $15, 30$, pas $6, 12, 18, 24, 30$.

```
int ppcmParMultiples(int a, b)
    int grand, petit, candidat
    if a < 0
        a = -a
    if b < 0
        b = -b
    if a == 0 OR b == 0
        candidat = 0
    else
        if a ≥ b
            grand = a
            petit = b
        else
            grand = b
            petit = a
        candidat = grand
        while candidat MOD petit ≠ 0
            candidat = candidat + grand
    return candidat
```

b) Avec le PGCD. Effectuer la division entière avant la multiplication réduit la taille du résultat intermédiaire. On traite le zéro avant la division.

```
int ppcmParPgcd(int a, b)
    int resultat
    if a < 0
        a = -a
```

```
    if b < 0
        b = -b
    resultat = 0
    if a ≠ 0 AND b ≠ 0
        resultat = (a DIV pgcd(a, b)) * b
    return resultat
```

Exercice 18 : Les nombres premiers

Syllabus, p. 55. Rejeter $n < 2$, accepter 2, puis rejeter les autres pairs. Pour un impair, tester uniquement les diviseurs impairs de 3 à $\lfloor \sqrt{n} \rfloor$ inclus. La condition `d ≤ n DIV d` équivaut à $d^2 \leq n$ pour $d > 0$, sans calculer d^2.

```
Boolean estPremier(int n)
    Boolean premier
    int d
    premier = (n == 2)
    if n > 2 AND n MOD 2 ≠ 0
        premier = true
        d = 3
        while premier AND d ≤ n DIV d
            premier = (n MOD d ≠ 0)
            d = d + 2
    return premier
```

Contrôles : $0, 1, 4, 9, 49$ ne sont pas premiers ; $2, 3, 97$ le sont. Tester la racine elle-même élimine notamment 49.

Exercice 19 : Encore des séries

Syllabus, p. 55. Chaque variante lit n et émet un seul terme par tour, y compris si le dernier groupe reste incomplet.

a) Énumérer de 1 au numéro du groupe.

```
void serieA()
    int n, i, longueur, valeur
    read n
    longueur = 1
    valeur = 1
    for i from 1 to n
        print valeur
        if valeur == longueur
            longueur = longueur + 1
            valeur = 1
        else
            valeur = valeur + 1
```

b) Répéter chaque valeur autant de fois qu'elle vaut.

```
void serieB()
    int n, i, valeur, restant
    read n
    valeur = 1
```

```
    restant = 1
    for i from 1 to n
        print valeur
        restant = restant - 1
        if restant == 0
            valeur = valeur + 1
            restant = valeur
```

c) Monter puis redescendre ; le sommet augmente après chaque descente. Le 1 qui termine une descente sert aussi de point de départ à la montée suivante : il n'est pas doublé.

```
void serieC()
    int n, i, sommet, valeur, pas
    read n
    sommet = 2
    valeur = 1
    pas = 1
    for i from 1 to n
        print valeur
        if pas == 1 AND valeur == sommet
            pas = -1
        else if pas == -1 AND valeur == 1
            sommet = sommet + 1
            pas = 1
        valeur = valeur + pas
```

Exercice 20 : Un peu de dessin

Syllabus, p. 55–56. Numéroter lignes i et colonnes j de 1 à n. Le tableau de décisions est dans l'ordre (a)–(h) ; choisir le numéro du modèle pour lancer une variante. Comme dans l'énoncé, les caractères d'une ligne sont accolés et seul `print` suivi du saut explicite termine la ligne.

```
void dessiner(int n, modele)
    int i, j
    Boolean estX
    for i from 1 to n
        for j from 1 to n
            estX = false
            if modele == 2
                estX = (i MOD 2 == 0)
            else if modele == 3
                estX = (j MOD 2 == 0)
            else if modele == 4
                estX = ((i + j) MOD 2 == 1)
            else if modele == 5
                estX = (i == j)
            else if modele == 6
                estX = (i == j OR i + j == n + 1)
            else if modele == 7
                estX = (i == 1 OR i == n OR j == 1 OR j == n)
            else if modele == 8
                estX = (j > n - i + 1)
            if estX
                print "X"
            else
                print "O"
```

```
        print "\n"

void dessinerTriangles()
    int n
    read n
    dessiner(n, 8)
```

Pour les autres variantes, remplacer 8 par 1 à 7 dans l'appel. En (h), la ligne i contient $i-1$ `X`, tous à droite ; pour $n=6$, la dernière ligne est donc `0XXXXX`. Pour $n=1$, les modèles diagonaux et le contour donnent `X` ; les cinq autres donnent `0`.

Exercice 21 : La frise

Syllabus, p. 56. Les motifs de six colonnes sont `0000XX`, `0XX0XX` et `0XX000`. La position dans le motif vaut $1+((j-1) \bmod 6)$. Un dernier motif partiel est imprimé normalement : on s'arrête à la longueur demandée.

```
void afficherFrise()
    int longueur, i, j, position
    Boolean estX
    read longueur
    for i from 1 to 3
        for j from 1 to longueur
            position = 1 + ((j - 1) MOD 6)
            if i == 1
                estX = (position == 5 OR position == 6)
            else if i == 2
                estX = (position MOD 3 ≠ 1)
            else
                estX = (position == 2 OR position == 3)
            if estX
                print "X"
            else
                print "0"
        print "\n"
```

Contrôle : pour la longueur 7, les trois lignes sont `0000XX0`, `0XX0XX0` et `0XX0000`.

Références

[1] ESI, HE2B, *Algorithmique I : syllabus 2026-2027*, Bruxelles, Belgique, 2026, pp. 49-56.